

LAPORAN PRAKTIKUM

▪ Identitas Praktikum

Nama MK : Praktikum Pemrograman Berorientasi Objek
Kode MK : CAK2KAB4
Bobot SKS : 3 SKS
Tempat : L-MM, Gedung IOT, lantai 3
Hari, tanggal : Selasa, 7 April 2026
Jam : 09:30-12:30 WIB
Topik praktikum : Modul-5

▪ Identitas Mahasiswa

Nama lengkap : Keishin Naufa Alfaridzhi
NIM : 103112400061
Program Studi : S-1 Teknik Informatika

▪ Algoritma & Penjelasan

Program ini merupakan simulasi pertarungan antara dua hero menggunakan konsep Pemrograman Berorientasi Objek (PBO). Algoritma program bekerja dengan cara berikut:

1. Inisialisasi Objek

Membuat dua objek hero dengan atribut yang berbeda (HP, Mana, Damage).

2. Validasi Atribut

Program memastikan nilai atribut berada dalam rentang yang valid saat objek dibuat (HP 0-100, Mana 0-100, Damage 0-15).

3. Proses Pertarungan

Kedua hero saling menyerang menggunakan basic attack dan skill attack sesuai urutan yang telah ditentukan di dalam program utama.

4. Pengurangan & Pemulihan HP

Setiap serangan akan mengurangi HP lawan sesuai dengan damage yang dimiliki. Hero juga dapat melakukan regenerasi HP jika kondisi memenuhi syara

5. Output Status

Program menampilkan status akhir kedua hero setelah serangkaian pertarungan berlangsung.

- **Kode Program & Penjelasan**
(Hero.java)

```
package Modul5.Unguided;

public class Hero {
    private String nama;
    private int hp, mana, damage;
    private Boolean checkHp, checkMana, checkDamage;
    private int maxHp;

    // Constructor
    public Hero(
        String nama,
        int hp,
        int mana,
        int damage
    ) {
        checkHp = hp >= 0 && hp <= 100;
        checkMana = mana >= 0 && mana <= 100;
        checkDamage = damage >= 0 && damage <= 15;
        if (
            checkHp &&
            checkMana &&
            checkDamage
        ) {
            this.nama = nama;
            this.hp = hp;
            this.mana = mana;
            this.damage = damage;
            this.maxHp = hp;
        } else {
            if (!checkHp && checkMana && checkDamage) {
                System.out.println("HP harus bernilai 0-100 !");
            } else if (!checkMana && checkHp && checkDamage) {
                System.out.println("Mana harus bernilai 0-100 !");
            } else if (!checkDamage && checkHp && checkMana) {
                System.out.println("Damage harus bernilai 0-15 !");
            } else {
                System.out.println("HP dan Mana harus bernilai 0-100 !");
                System.out.println("Damage harus bernilai 0-15 !");
            }
        }
    }

    public void basicAttack(Hero target) {
        target.hp -= this.damage;
        if (target.hp < 0) {
            target.hp = 0;
        }
        System.out.println("[ "+this.nama+" ] Melancarkan Basic Attack ke " +
target.nama + " (Damage: " + this.damage + ")");
        System.out.println();
    }
}
```

```

public void skillAttack(Hero target) {
    if (this.mana >= 25) {
        target.hp -= this.damage*2;
        this.mana -= 25;

        if (target.hp < 0) {
            target.hp = 0;
        }
        System.out.println("[ "+this.nama+" ] Melancarkan Skill ke " +
target.nama + " (Damage: " + this.damage*2 + ")");
        System.out.println();
    } else {
        System.out.println("[ "+this.nama+" ] Mana tidak cukup untuk melancarkan
skill.");
        System.out.println();
    }
}

public void regenHp() {
    if (this.hp < maxHp && this.hp > 0) {
        this.hp += 15;
        if (this.hp > 100) {
            this.hp = maxHp;
        }
        System.out.println("[ "+this.nama+" ] Regenerasi, +15 Hp");
        System.out.println();
    } else {
        System.out.println("[ "+this.nama+" ] Tidak dapat regenerasi (mati).");
        System.out.println();
    }
    // return target.hp;
}

public void attributeStatus() {
    System.out.println("Nama : " + this.nama);
    System.out.println("Current Hp / Max Hp : " + this.hp + " / " + maxHp);
    System.out.println("Mana : " + this.mana);
    System.out.println("Damage: " + this.damage);
    if (this.hp == 0) {
        System.out.println("Status: DEAD");
    } else if (this.hp > 0) {
        System.out.println("Status: ALIVE");
    }
    System.out.println();
}
}

```

Penjelasan:

Class **Hero** merupakan class utama yang merepresentasikan hero dalam game Mobile Legends: Bang Bang dengan menerapkan konsep enkapsulasi.

1. Inisialisasi Variabel

Class ini memiliki atribut private yang menyimpan informasi penting seperti nama hero, health points (hp), mana, dan damage yang dihasilkan. Terdapat juga atribut **maxHp** yang menyimpan nilai maksimum HP untuk keperluan regenerasi.

2. Constructor & Validasi

Constructor **Hero** menerima empat parameter dan melakukan validasi terhadap setiap nilai atribut sebelum menginisialisasi objek. Validasi menggunakan operator logika AND (**&&**) untuk memastikan hp dan mana berada dalam rentang 0-100, serta damage dalam rentang 0-15. Jika terdapat nilai yang tidak valid, program akan menampilkan pesan error yang spesifik.

3. basicAttack(Hero target)

Digunakan untuk melakukan *basic attack* kepada hero lawan. Method ini mengurangi HP target sebesar damage hero penyerang, dengan perlindungan agar HP tidak bernilai negatif dengan mengatur nilai minimum menjadi 0.

4. skillAttack(Hero target)

Merupakan *skill attack* yang membutuhkan konsumsi 25 mana. Jika mana mencukupi, hero akan memberikan damage dua kali lipat dari damage normal dan mengurangi mana sebesar 25 poin. Jika mana tidak mencukupi, program akan menampilkan pesan bahwa mana tidak cukup.

5. regenHp()

Digunakan untuk memulihkan HP hero sebesar 15 poin selama hero masih hidup (HP > 0 dan HP < maxHp). Method ini memiliki proteksi agar HP tidak melebihi nilai maksimum (100).

6. attributeStatus()

Menampilkan informasi lengkap hero seperti nama, current HP/max HP, mana, damage, dan status **ALIVE/DEAD**.

(Main.java)

```
package Modul5.Unguided;

public class Main {
    public static void main(String[] args) {
        Hero jawhead = new Hero("jawhead", 72, 51, 15);
        Hero xborg = new Hero("xborg", 89, 100, 12);

        jawhead.basicAttack(xborg);
        jawhead.skillAttack(xborg);
        jawhead.skillAttack(xborg);
        xborg.skillAttack(jawhead);
        xborg.regenHp();
        jawhead.skillAttack(xborg);
        jawhead.basicAttack(xborg);
        jawhead.basicAttack(xborg);
    }
}
```

```
xborg.regenHp();

xborg.attributeStatus();
jawhead.attributeStatus();
}
}
```

Penjelasan:

Class **Main** merupakan class yang mengontrol jalannya pertarungan antara dua hero.

1. Pembuatan Objek

Di dalam method **main()**, pertama dibuat dua objek **Hero** yaitu **jawhead** (HP 72, mana 51, damage 15) dan **xborg** (HP 89, mana 100, damage 12).

2. Alur Battle

Dilakukan serangkaian pertarungan sesuai urutan yang telah ditentukan. Pertama, jawhead melakukan basic attack ke xborg, kemudian jawhead menggunakan skill attack sebanyak dua kali berturut-turut. Setelah itu, xborg membalas dengan skill attack ke jawhead dan melakukan regenerasi HP. Selanjutnya jawhead kembali menyerang dengan skill attack dan basic attack.

3. Hasil Akhir

Di akhir pertarungan, program memanggil method **attributeStatus()** untuk menampilkan atribut lengkap kedua hero, guna melihat hasil battle yang telah dilakukan.

- **Hasil Running Program**

```
[jawhead] Melancarkan Basic Attack ke xborg (Damage: 15)
[jawhead] Melancarkan Skill ke xborg (Damage: 30)
[jawhead] Melancarkan Skill ke xborg (Damage: 30)
[xborg] Melancarkan Skill ke jawhead (Damage: 24)
[xborg] Regenerasi, +15 Hp
[jawhead] Mana tidak cukup untuk melancarkan skill.
[jawhead] Melancarkan Basic Attack ke xborg (Damage: 15)
[jawhead] Melancarkan Basic Attack ke xborg (Damage: 15)
[xborg] Tidak dapat regenerasi (mati).

Nama : xborg
Current Hp / Max Hp : 0 / 89
Mana : 75
Damage: 12
Status: DEAD

Nama : jawhead
Current Hp / Max Hp : 48 / 72
Mana : 1
Damage: 15
Status: ALIVE
```

- **Link Program Praktikum (Github)**

[kukingkux/PRAKTIKUM-PBO](https://github.com/kukingkux/PRAKTIKUM-PBO)